APARTMENT MAINTENANCE REQUEST MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

HEMACHANDIRAN A

(240701188)

in partial fulfilment of the End Semester Examination for the course

CB23F35 – LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "APARTMENT MAINTENANCE REQUEST MANAGEMENT SYSTEM" is the bonafide work of HEMACHANDIRAN A (240701188), submitted for the course CB23F35 – Low-Code Application Development using Zoho Creator during the academic year 2026–2027, Semester V. The implementation and evidence described in this report must be verified against the submitted Zoho Creator application before certification.

Faculty and Examination Signatures

HEAD OF THE DEPARTMENT	COURSE FACULTY
Dr. Manoranjini Alwin	Jinu Sophia
Department of Computer Science and Engineering	Assistant Professor
Signature: _____	Signature: _____

Submitted for End Semester Examination on: _____

INTERNAL EXAMINER EXTERNAL EXAMINER

DECLARATION

I, HEMACHANDIRAN A (240701188), declare that the report entitled "APARTMENT MAINTENANCE REQUEST MANAGEMENT SYSTEM" describes the requirements, design and proposed implementation of an apartment maintenance request application using Zoho Creator. Any final claim of implemented functionality, test execution or deployment will be supported by actual application evidence.

I acknowledge the use of platform documentation, academic guidance and other referenced resources. The report is prepared for the course CB23F35 – Low-Code Application Development using Zoho Creator.

Place: Chennai Date: _____

SIGNATURE OF THE STUDENT

ACKNOWLEDGEMENT

I express my gratitude to the management of Rajalakshmi Engineering College for providing the academic facilities and learning environment necessary for this project.

I sincerely thank the Principal, Head of the Department of Computer Science and Engineering, and the teaching faculty for their support and encouragement.

I extend my thanks to the course faculty, Aswana Lal, Assistant Professor, for guidance in understanding low-code application development and project documentation.

I also acknowledge Zoho Corporation for its low-code application development platform and documentation. I thank my family and friends for their continued encouragement.

ABSTRACT

Apartment communities receive maintenance requests concerning plumbing, electrical faults, common-area cleaning, water supply, lift servicing, security equipment and other shared facilities. In a manual system, residents may contact the manager through phone calls, paper registers or messaging applications. Such requests can be overlooked, duplicated or closed without a traceable service record.

The Apartment Maintenance Request Management System is designed as a low-code application on Zoho Creator to register residents and apartments, receive maintenance requests, allocate tasks to technicians, track service progress, record completion details and collect resident feedback. A structured data model connects resident, unit, request, technician, assignment, work update and feedback records.

The proposed application supports status values such as Submitted, Under Review, Assigned, In Progress, On Hold and Resolved. Role-based reports can help the administrator identify pending requests, assigned jobs, overdue tasks and completed work. Workflow validation and Deluge scripts may be used to check required fields, assign default status and record resolution information.

This report presents the system study, requirements, architecture, form specifications, workflow design, implementation plan, test cases, security controls, deployment considerations and future enhancements. Application screenshots and test outcomes are not represented as verified unless actual workspace evidence is supplied.

Keywords: Low-Code, Zoho Creator, Apartment Management, Maintenance Requests, Work Orders, Deluge, Reports.

PROJECT AT A GLANCE

Project Summary

Attribute	Details
Project Title	Apartment Maintenance Request Management System
Student	HEMACHANDIRAN A – 240701188
Course	CB23F35 – Low-Code Application Development using Zoho Creator
Application Domain	Residential property and facility maintenance
Platform	Zoho Creator (proposed)
Core Forms	Resident Details, Apartment Unit, Maintenance Request, Technician Details, Work Assignment, Service Update, Resident Feedback
Reports	All Requests, Pending Requests, Assigned Jobs, Resolved Requests, Technician Workload
Roles	Resident, Apartment Administrator, Maintenance Technician
Automation	Status defaults, validation, assignment and follow-up (planned)
Evidence Status	Proposed design; actual implementation to be verified

TABLE OF CONTENTS

CHAPTER 1 – INTRODUCTION.....12
1.1 Background 12
1.2 Problem Statement 12
1.3 Objectives.....12
1.4 Scope of the Project 12
1.5 Justification for Low-Code Development 13
1.6 Organisation of the Report..... 13
CHAPTER 2 – LITERATURE SURVEY AND SYSTEM STUDY 14
2.1 Study of the Existing System 14
2.2 Limitations of the Existing System 14
2.3 Low-Code versus Traditional Development 14
2.4 Literature Survey.....14
2.5 Emerging Trends..... 15
2.6 Proposed System 15
CHAPTER 3 – REQUIREMENT ANALYSIS 16
3.1 Stakeholders and User Roles.....16
3.2 Functional Requirements.....16
3.3 Non-Functional Requirements 16
3.4 Hardware and Software Requirements.....16
3.5 Use Case Model 16
CHAPTER 4 – SYSTEM DESIGN 21
4.1 System Architecture 21
4.2 Data Model..... 21
4.3 Form Design..... 21
4.4 Process and Workflow Design 21
4.5 User Interface and Navigation Design 21
4.6 Access Control Design..... 21
CHAPTER 5 – IMPLEMENTATION 26
5.1 Application Setup..... 26
5.2 Forms and Data Capture.....26
5.3 Reports 26
5.4 Pages and Dashboards..... 26
5.5 Deluge Scripting.....26
5.6 Workflows and Automation 26

5.7 Integrations.....27
5.8 User Management and Role-Based Access.....27
5.9 AI and Analytics Features..... 27
5.10 Pro-Code Extension 27
5.11 Mobile and Portal Access.....27
CHAPTER 6 – TESTING AND DEBUGGING.....30
6.1 Testing Strategy 30
6.2 Test Cases 30
6.3 Debugging Deluge Scripts 30
6.4 Defect Log.....30
6.5 User Acceptance Testing.....30
CHAPTER 7 – DEPLOYMENT, SECURITY AND GOVERNANCE 33
7.1 Environments and Deployment Strategy.....33
7.2 Performance Optimisation.....33
7.3 Security Features..... 33
7.4 Governance and Administration.....33
CHAPTER 8 – RESULTS AND DISCUSSION 35
8.1 Application Output..... 35
8.2 Sample Scenario..... 35
8.3 Achievement of Objectives 35
8.4 Existing versus Proposed Application.....35
8.5 Limitations and Low-Code Reflection 35
CHAPTER 9 – CONCLUSION AND FUTURE ENHANCEMENTS 37
9.1 Conclusion.....37
9.2 Future Enhancements 37
9.3 Learning Outcomes and Career Relevance 37
REFERENCES.....38
APPENDIX A..... 39
APPENDIX B..... 40
APPENDIX C..... 41
APPENDIX D..... 42
APPENDIX E 43
APPENDIX F.....44

LIST OF TABLES

Faculty and Examination Signatures 2
Project Summary 6
Table 2.1: Existing System versus Proposed System 15
Table 3.1: Stakeholders and User Roles 17
Table 3.2: Functional Requirements 17
Table 3.3: Non-Functional Requirements 18
Table 3.4: Hardware and Software Requirements 18
Use Case: Submit Maintenance Request 19
Use Case: Assign Technician 19
Use Case: Record Work Update 19
Use Case: Resolve Request 20
Table 4.1: Field Specification – Resident Details 22
Table 4.2: Field Specification – Apartment Unit 23
Table 4.3: Field Specification – Maintenance Request 23
Table 4.4: Field Specification – Technician Details 23
Table 4.5: Field Specification – Work Assignment 24
Table 4.6: Field Specification – Service Update 24
Table 4.7: Field Specification – Resident Feedback 24
Table 5.1: Proposed Reports 27
Table 5.2: Planned Deluge Scripts 28
Table 6.1: Planned Functional Test Cases 31
Table 6.2: Defect Log Template 31
Table 7.1: Security Control Plan 33
Table 8.1: Objective Mapping 36
Table 8.2: Existing versus Proposed Application 36
Table A.1: Feature Coverage Checklist 39
Table B.1: Course Outcome Mapping 40
Table E.1: Evaluation Rubric 43
Table F.1: Viva Questions 44

LIST OF FIGURES

Figure 2.1: As-Is Maintenance Request Process 15

Figure 3.1: Use Case Diagram (Textual Representation) 18

Figure 4.1: Application Architecture 22

Figure 4.2: Entity Relationship Overview 22

Figure 4.3: Maintenance Request Workflow 25

Figure 5.1: Proposed Application Navigation 28

Figure 5.2: Request Submission Form Layout 28

Figure 5.3: Pending Requests Report Layout 28

Figure 6.1: Testing and Debugging Cycle 32

Figure 7.1: Deployment Workflow 34

Figure 8.1: Example Request-to-Resolution Scenario 36

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
DBMS	Database Management System
FR	Functional Requirement
NFR	Non-Functional Requirement
UI	User Interface
UAT	User Acceptance Testing
RBAC	Role-Based Access Control
SLA	Service Level Agreement
SMS	Short Message Service
ZC	Zoho Creator

CHAPTER 1

INTRODUCTION

1.1 Background

Apartment buildings depend on coordinated maintenance of private units and shared infrastructure. A resident may need a leaking tap repaired, an electrical circuit checked, a lift fault inspected or a corridor cleaned. A request must reach the responsible staff member and remain visible until closure.

Residential buildings depend on coordinated maintenance of private units and shared infrastructure. A resident may need a leaking tap repaired, an electrical circuit checked, a lift fault inspected or a corridor cleaned. A service request must reach the responsible staff member and remain visible until closure.

1.2 Problem Statement

Requests made by phone, handwritten registers or informal chat groups do not provide a consistent reference number, status history or assigned technician. Administrators may have difficulty measuring response time, tracking repeated problems or verifying that work was completed.

Requests made by phone, handwritten registers or informal chat groups do not provide a consistent reference number, status history or assigned technician. Administrators may have difficulty measuring response time, tracking repeated problems or verifying that work was completed.

1.3 Objectives

The project aims to centralise request submission, classify maintenance issues, assign tasks, record progress, track resolution and provide role-specific reports. Additional objectives include reducing duplicate entries, improving communication and maintaining an auditable history.

1.4 Scope of the Project

The scope covers resident and apartment registration, request creation, technician assignment, progress updates, closure and feedback. It excludes payment collection, legal dispute handling, automated safety decisions and direct control of building equipment.

1.5 Justification for Low-Code Development

Zoho Creator offers visual form creation, lookup relationships, reports and workflow configuration. This suits structured maintenance processes and enables rapid prototyping while still requiring testing, permission design and administrative oversight.

1.6 Organisation of the Report

The following chapters discuss the existing process, requirements, system design, low-code implementation, testing, deployment, results and future enhancements.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

A typical apartment association receives complaints through a register, a telephone number or an instant messaging group. The administrator identifies the type of problem, contacts a worker and later asks whether the repair was completed.

This process relies on personal follow-up and does not automatically preserve a searchable record of each stage. When personnel change, the history of repeated defects may be lost.

2.2 Limitations of the Existing System

Requests may lack clear descriptions, unit numbers or contact details. Informal communication may result in repeated submissions, forgotten assignments and disagreements about whether the issue was resolved.

Managers may struggle to distinguish open, urgent, overdue and completed requests without manually reading messages or files.

2.3 Low-Code versus Traditional Development

A conventional web application may require separate frontend, backend, database, hosting and authentication components. A low-code platform provides integrated tools for common data-entry and workflow patterns.

Low-code accelerates development but platform restrictions, subscription limits and vendor dependence must be considered.

2.4 Literature Survey

Facility management systems commonly use work orders, service categories, technician assignments, status transitions and maintenance history. Helpdesk systems similarly use ticket identifiers, priorities and resolution records.

Low-code development literature emphasises rapid prototyping, reuse and business-user participation, while identifying governance, security and integration challenges.

2.5 Emerging Trends

Modern maintenance systems may offer mobile reporting, photo attachments, automated notifications, analytics, preventive maintenance schedules and service-level monitoring. Such features should be implemented incrementally and tested with realistic scenarios.

2.6 Proposed System

The proposed system centralises seven linked forms and produces filtered reports. Each maintenance request has a unique reference, issue category, priority, location, submission date and status. Assignments connect requests to technicians and service updates preserve the work history.

Table 2.1: Existing System versus Proposed System

Parameter	Existing Process	Proposed Application
Submission	Calls or paper entries	Structured request form
Tracking	Manual follow-up	Request status report
Assignment	Phone or message	Linked work assignment
History	Scattered records	Searchable service updates
Closure	Informal confirmation	Resolution and feedback records

Figure 2.1: As-Is Maintenance Request Process



CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Residents submit issues and check permitted status. The apartment administrator reviews, prioritises and assigns work. Technicians view assigned tasks and record updates. A property committee may review aggregated operational reports.

3.2 Functional Requirements

The application must register residents and apartment units, capture maintenance requests, assign technicians, update progress, record resolution, accept feedback and generate filtered reports.

3.3 Non-Functional Requirements

The application should be secure, usable on common browsers, responsive on mobile devices, maintainable and reliable for expected request volumes. Permissions must prevent residents from viewing other households' private details.

3.4 Hardware and Software Requirements

Development requires a Zoho Creator account and a modern web browser. End users require internet access through a desktop or mobile device. Deluge scripting may be used for additional validation.

3.5 Use Case Model

Core use cases include Register Resident, Submit Request, Review Request, Assign Technician, Update Work, Close Request, View Reports and Submit Feedback.

Table 3.1: Stakeholders and User Roles

Role	Description	Main Responsibility
Resident	Occupant of registered apartment	Submit and track own requests
Administrator	Apartment management representative	Review, assign, report and close
Technician	Maintenance service worker	Update assigned work
Committee	Authorised oversight group	View approved summary reports

Table 3.2: Functional Requirements

ID	Requirement	Priority
FR-01	Maintain resident details	High
FR-02	Maintain apartment unit details	High
FR-03	Submit maintenance request	High
FR-04	Capture issue category and priority	High
FR-05	Track request status	High
FR-06	Assign request to technician	High
FR-07	Record service progress	High
FR-08	Record resolution date and remarks	High
FR-09	Collect resident feedback	High
FR-10	Show pending and resolved reports	High
FR-11	Filter technician assignments	Medium
FR-12	Restrict records by user role	Medium

Table 3.3: Non-Functional Requirements

Area	Requirement
Security	Authenticated accounts and least-privilege permissions
Usability	Clear field labels and readable status values
Performance	Efficient filtered reports
Reliability	Consistent updates and backup planning
Privacy	Resident records visible only to authorised roles
Maintainability	Documented fields, scripts and workflows

Table 3.4: Hardware and Software Requirements

Component	Specification
Platform	Zoho Creator
Client	Modern desktop or mobile browser
Network	Internet connection
Automation	Deluge scripting and Creator workflows
Storage	Managed platform database subject to subscription limits

Figure 3.1: Use Case Diagram (Textual Representation)

Resident: Submit and view own requests
Administrator: Review, assign and close requests
Technician: View tasks and update work
Committee: Review approved summary reports

3.5.1 Submit Maintenance Request

Use Case: Submit Maintenance Request

Item	Description
Actor	Resident
Pre-condition	Authenticated user and relevant record exists
Main Flow	Select unit, category, description and priority; submit; system creates a request reference.
Post-condition	Record saved and visible in authorised reports

3.5.2 Assign Technician

Use Case: Assign Technician

Item	Description
Actor	Administrator
Pre-condition	Authenticated user and relevant record exists
Main Flow	Review pending request, choose technician and target date; save assignment; update status.
Post-condition	Record saved and visible in authorised reports

3.5.3 Record Work Update

Use Case: Record Work Update

Item	Description
Actor	Technician
Pre-condition	Authenticated user and relevant record exists
Main Flow	Open assigned job, enter progress and remarks, optionally attach authorised evidence.
Post-condition	Record saved and visible in authorised reports

3.5.4 Resolve Request

Use Case: Resolve Request

Item	Description
Actor	Administrator
Pre-condition	Authenticated user and relevant record exists
Main Flow	Review completed work, record resolution details and mark the request Resolved.
Post-condition	Record saved and visible in authorised reports

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system uses a browser presentation layer, Zoho Creator forms and reports, workflow rules and a relational application data layer. All requests should pass through authenticated, role-specific interfaces.

4.2 Data Model

Resident Details links to Apartment Unit. Maintenance Request references the resident and unit. Work Assignment references a request and a technician. Service Update references an assignment or request. Resident Feedback references the resolved request.

4.3 Form Design

Seven proposed forms define the main data capture operations. Field types include single line, multi line, dropdown, date, date-time, lookup, number, email and file upload.

4.4 Process and Workflow Design

The status flow starts with Submitted and may progress through Under Review, Assigned, In Progress, On Hold and Resolved. A request may be reopened under a documented policy.

4.5 User Interface and Navigation Design

Residents see Submit Request and My Requests. Administrators see All Requests, Assignments, Technicians and Reports. Technicians see My Assigned Work and Update Progress.

4.6 Access Control Design

Role permissions should restrict personal information, technician notes and administrative actions. Residents should only view their own requests and feedback records.

Figure 4.1: Application Architecture

Web Browser / Mobile Interface
Zoho Creator Forms and Reports
Deluge Workflow and Validation
Related Application Records

Figure 4.2: Entity Relationship Overview

Apartment Unit → Resident Details
Resident Details → Maintenance Request
Maintenance Request → Work Assignment
Technician Details → Work Assignment
Work Assignment → Service Update
Maintenance Request → Resident Feedback

Table 4.1: Field Specification – Resident Details

Field Name	Type	Validation / Remarks
Resident_ID	Single Line	Required, unique
Full_Name	Single Line	Required
Contact_Number	Phone	Required
Email	Email	Valid format
Apartment_Unit	Lookup	References unit

Table 4.2: Field Specification – Apartment Unit

Field Name	Type	Validation / Remarks
Unit_Number	Single Line	Required, unique
Block	Single Line	Required
Floor	Number	Non-negative
Occupancy_Status	Dropdown	Active / Vacant

Table 4.3: Field Specification – Maintenance Request

Field Name	Type	Validation / Remarks
Request_ID	Single Line	Unique reference
Resident	Lookup	Required
Apartment_Unit	Lookup	Required
Issue_Category	Dropdown	Required
Description	Multi Line	Required
Priority	Dropdown	Low / Medium / High
Status	Dropdown	Default Submitted
Request_Date	Date	Required

Table 4.4: Field Specification – Technician Details

Field Name	Type	Validation / Remarks
Technician_ID	Single Line	Unique
Name	Single Line	Required
Specialisation	Dropdown	Plumbing / Electrical / General
Phone	Phone	Valid number
Availability	Dropdown	Available / Busy

Table 4.5: Field Specification – Work Assignment

Field Name	Type	Validation / Remarks
Maintenance_Request	Lookup	Required
Technician	Lookup	Required
Assigned_Date	Date	Required
Target_Date	Date	Not before assignment
Assignment_Status	Dropdown	Assigned / In Progress / Done

Table 4.6: Field Specification – Service Update

Field Name	Type	Validation / Remarks
Assignment	Lookup	Required
Update_Date	Date	Required
Progress_Notes	Multi Line	Required
Work_Status	Dropdown	Controlled values
Completion_Photo	File Upload	Optional

Table 4.7: Field Specification – Resident Feedback

Field Name	Type	Validation / Remarks
Maintenance_Request	Lookup	Required
Rating	Number	1 to 5
Feedback_Notes	Multi Line	Optional
Feedback_Date	Date	Required

Figure 4.3: Maintenance Request Workflow

Submit Request
Review Category and Priority
Assign Technician
Record Service Progress
Verify Completion
Resolve and Collect Feedback

CHAPTER 5

IMPLEMENTATION

5.1 Application Setup

Create a Zoho Creator application with the project title and configure the forms, navigation and workspace permissions. Begin with fictitious data and verify field link names before adding Deluge scripts.

5.2 Forms and Data Capture

Build Apartment Unit and Resident Details first, followed by Maintenance Request and Technician Details. Configure Work Assignment, Service Update and Resident Feedback after their parent forms exist.

5.3 Reports

Reports should include All Maintenance Requests, Pending Requests, Assigned Requests, In Progress Requests, Resolved Requests, Technician Workload and Feedback Summary.

5.4 Pages and Dashboards

An optional administrator dashboard can show open requests, overdue jobs, assignments by technician and recent resolutions. The dashboard should not expose private contact information unnecessarily.

5.5 Deluge Scripting

Suggested scripts set default status, validate dates, prevent empty descriptions, update assignment status and flag overdue requests. Each script must be tested against actual form link names.

5.6 Workflows and Automation

On submission, a request can receive a default status. On assignment, a status change can be recorded. On completion, a review task may be created. Notifications require approved recipient and message configuration.

5.7 Integrations

Email or messaging integrations may be added after consent and institutional approval. No integration is assumed to be implemented.

5.8 User Management and Role-Based Access

Residents, administrators and technicians should use separate authenticated accounts. Record-level restrictions must be verified with distinct test users.

5.9 AI and Analytics Features

Analytics can summarise requests by category, building, priority and status. AI-based routing may be considered later, but should not replace administrator review.

5.10 Pro-Code Extension

Custom APIs or widgets may extend the application when visual configuration and Deluge are insufficient. These extensions require additional security and maintenance.

5.11 Mobile and Portal Access

Mobile-friendly forms can help residents submit issues from their apartments and technicians update jobs on site.

Table 5.1: Proposed Reports

Report	Filter / Purpose
All Requests	All authorised maintenance tickets
Pending Requests	Status Submitted or Under Review
Assigned Jobs	Technician and target date
In Progress	Active service work
Resolved Requests	Resolution date and category
Technician Workload	Assignments by technician
Feedback Summary	Ratings and comments for resolved work

Figure 5.1: Proposed Application Navigation

Resident: Submit Request / My Requests
Administrator: Requests / Assignments / Reports
Technician: Assigned Work / Progress Updates

Figure 5.2: Request Submission Form Layout

Apartment and Resident Lookup
Category, Priority and Description
Optional Photo Attachment
Submit and Generate Request ID

Figure 5.3: Pending Requests Report Layout

Request ID / Unit / Category
Submitted Date / Priority / Status
Assigned Technician / Target Date

Table 5.2: Planned Deluge Scripts

Script ID	Trigger	Purpose
DS-01	Request On Load	Set default status Submitted
DS-02	Request On Validate	Require issue description
DS-03	Assignment On Validate	Check target date
DS-04	Assignment On Success	Update linked request status
DS-05	Service Update On Success	Update progress / completion

Code 5.1: Default Request Status

```
if(input.Status == null)
{
input.Status = "Submitted";
}
```

Code 5.2: Validate Issue Description

```
if(input.Description == null)
{
alert "Please describe the maintenance issue.";
cancel submit;
}
```

Code 5.3: Validate Assignment Date

```
if(input.Target_Date < input.Assigned_Date)
{
alert "Target date cannot be earlier than assignment.";
cancel submit;
}
```

CHAPTER 6

TESTING AND DEBUGGING

6.1 Testing Strategy

Testing should cover form validation, lookup consistency, report filtering, status transitions, role permissions and workflow behaviour. All records used in the academic demonstration should be fictitious.

6.2 Test Cases

The test matrix below contains planned tests and expected outcomes. It is not an assertion that the tests were executed. Actual results and pass/fail statuses must be entered after running the application.

6.3 Debugging Deluge Scripts

Use the platform's available execution logs and test records to investigate validation failures, incorrect lookups and unexpected status updates. Confirm field link names and event types.

6.4 Defect Log

Each defect should record the affected module, steps to reproduce, expected result, actual result, severity, assignee and retest outcome.

6.5 User Acceptance Testing

A resident, administrator and technician should each test representative tasks using separate accounts. Feedback should be recorded and unresolved issues addressed before production use.

Table 6.1: Planned Functional Test Cases

ID	Test Description	Expected Result
TC-01	Create valid resident	Record saved
TC-02	Missing resident name	Validation prevents save
TC-03	Duplicate unit number	Duplicate rejected
TC-04	Submit valid request	Request saved as Submitted
TC-05	Missing issue description	Validation message
TC-06	Select request category	Category stored
TC-07	Assign technician	Assignment created
TC-08	Invalid target date	Validation prevents save
TC-09	Update service progress	Update stored
TC-10	Mark work resolved	Resolved status shown
TC-11	Filter pending requests	Only pending shown
TC-12	Filter resolved requests	Only resolved shown
TC-13	Resident accesses other unit	Access denied
TC-14	Technician opens unassigned job	Access denied
TC-15	Submit feedback	Feedback linked
TC-16	Invalid rating	Validation prevents save
TC-17	View technician workload	Correct assignment counts
TC-18	Check overdue filter	Past-target open jobs listed
TC-19	Cancel request	Authorised status change

Table 6.2: Defect Log Template

Defect ID	Issue	Status
D-01	To be recorded after testing	Not evaluated
D-02	To be recorded after testing	Not evaluated

Figure 6.1: Testing and Debugging Cycle

Prepare Sample Data
Execute Test Case
Compare Expected and Actual
Log Defect
Correct and Retest

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Develop the application in a controlled workspace. Validate forms, reports and permissions before any production deployment. Promotion should occur only after approval from the apartment association or authorised owner.

7.2 Performance Optimisation

Limit reports to necessary columns and apply filters. Avoid redundant workflow operations. Review response times using representative data volumes.

7.3 Security Features

Authentication, role-based permissions, record restrictions and secure file handling are essential. Contact details, unit numbers and resident requests must not be exposed through public reports.

7.4 Governance and Administration

Assign an application owner and backup administrator. Maintain a change log, define retention and backup rules, and remove access promptly when residents or staff leave.

Table 7.1: Security Control Plan

Control	Purpose
Authentication	Prevent anonymous access
Role permissions	Restrict resident, technician and administrator actions
Record restrictions	Prevent cross-unit data exposure
Attachment handling	Restrict access to uploaded photos
Audit review	Track administrative changes
Backup and recovery	Support restoration after errors

Figure 7.1: Deployment Workflow

Development
Role-Based Testing
Administrative Approval
Production
Monitoring and Maintenance

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

The intended output is a working maintenance-request register with linked resident, unit, technician, assignment, service update and feedback records. Actual screenshots should be inserted after the Zoho Creator application is configured.

8.2 Sample Scenario

A resident of Block B, Unit 204 submits a plumbing request for a leaking kitchen tap. The administrator reviews the request, assigns a plumber and sets a target date. The technician records progress and the administrator closes the request after confirmation.

8.3 Achievement of Objectives

The proposed design addresses centralised submission, tracking, assignment, resolution and reporting. These objectives should be marked achieved only after implementation and test evidence are available.

8.4 Existing versus Proposed Application

The proposed system provides searchable records and explicit status tracking compared with scattered paper and chat messages. Improvements in response time must be measured rather than assumed.

8.5 Limitations and Low-Code Reflection

The application depends on internet connectivity, platform availability and user adoption. Complex scheduling, payment integration and advanced analytics may require further configuration or custom development.

Figure 8.1: Example Request-to-Resolution Scenario

Resident: Plumbing Issue, Block B / Unit 204
Administrator: Review and Assign
Technician: Work Update
Administrator: Verify and Resolve
Resident: Feedback

Table 8.1: Objective Mapping

Objective	Proposed Evidence
Centralised submission	Maintenance Request form
Assignment tracking	Work Assignment and Technician Details
Progress visibility	Service Update and status reports
Resolution history	Resolved Requests report
Resident feedback	Resident Feedback form

Table 8.2: Existing versus Proposed Application

Parameter	Existing	Proposed
Request history	Messages / registers	Linked records
Status visibility	Informal follow-up	Status-based reports
Technician allocation	Phone assignment	Assignment form
Closure	Verbal confirmation	Resolution record
Feedback	Unstructured	Feedback form

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Apartment Maintenance Request Management System proposes a structured, low-code approach to receiving, assigning, monitoring and resolving residential maintenance requests. The design integrates core data forms, lookup relationships, status-based reports and controlled workflows.

9.2 Future Enhancements

Future improvements may include resident mobile notifications, preventive maintenance scheduling, vendor management, repair cost tracking, service-level analytics, QR-based equipment identification and integration with approved apartment portals.

9.3 Learning Outcomes and Career Relevance

The project develops skills in requirements analysis, relational data design, Zoho Creator form configuration, workflow planning, Deluge validation, test design and secure application administration.

REFERENCES

[1] Zoho Corporation, "Zoho Creator," https://www.zoho.com/creator/.

[2] Zoho Corporation, "Deluge Scripting Language," https://www.zoho.com/deluge/.

[3] Zoho Corporation, "Zoho Creator Help and Documentation," https://www.zoho.com/creator/help/.

[4] OWASP Foundation, "Application Security Verification Standard," https://owasp.org/.

[5] NIST, "Cybersecurity Framework 2.0," https://www.nist.gov/cyberframework.

[6] Rajalakshmi Engineering College, CB23F35 course materials.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Feature	Project Use	Verification
Forms	Seven related forms	To verify
Lookup fields	Resident, request and assignment links	To verify
Reports	Pending, assigned and resolved	To verify
Workflows	Status changes and validation	To verify
Deluge	Form-event logic	To verify
Role permissions	Resident, technician, admin	To verify
Dashboard	Operational summary	Optional
Notifications	Assignment and resolution messages	Optional

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Course Outcome Mapping

Learning Outcome	Report Evidence
Requirements	Chapters 2–3
Application data modelling	Chapter 4
Forms and reports	Chapter 5
Workflow and Deluge	Chapter 5
Testing	Chapter 6
Security and deployment	Chapter 7
Evaluation	Chapters 8–9

APPENDIX C – SAMPLE DELUGE SOURCE CODE

The following are sample Deluge patterns. Verify all field link names and event compatibility in the actual application before using them.

```
// Default status
if(input.Status == null)
{
input.Status = "Submitted";
}
// Required description
if(input.Description == null)
{
alert "Enter the issue description.";
cancel submit;
}
// Assignment date
if(input.Target_Date < input.Assigned_Date)
{
alert "Choose a valid target date.";
cancel submit;
}
```

APPENDIX D – USER MANUAL

D.1 Resident Login

Open the authorised apartment maintenance portal and sign in with your individual account.

D.2 Submit Request

Choose the apartment unit, issue category, priority and description, then submit.

D.3 Track Request

Open My Requests to review status and any visible updates.

D.4 Administrator Assignment

Open Pending Requests, select a technician and save the assignment.

D.5 Technician Update

Open assigned work and enter progress and completion notes.

D.6 Close Request

Verify the completed work and update the status to Resolved.

D.7 Feedback

After closure, the resident may submit a rating and comment.

APPENDIX E – EVALUATION RUBRIC

Table E.1: Evaluation Rubric

Criterion	Maximum Marks
Problem and objectives	10
Requirements	10
Data model	15
Forms and reports	20
Workflows and scripting	15
Testing	15
Security and documentation	10
Viva	5
Total	100

APPENDIX F – VIVA-VOCE RECORD SHEET

Student: HEMACHANDIRAN A

Register Number: 240701188

Project: Apartment Maintenance Request Management System

Table F.1: Viva Questions

Question	Examiner Notes
What is low-code application development?	
Why use Zoho Creator for apartment maintenance?	
Explain the seven forms and lookup relationships.	
How are requests assigned to technicians?	
What does a Deluge workflow do?	
How is resident data protected?	
What are the main test cases?	
What future enhancements are possible?	

Internal Examiner: _____ External Examiner: _____